

# Research Note: Low-Level Design (LLD) Practice & Evaluation Platform

**Author:** Engineering Assignment Submission | **Target Track:** CipherSchools 2-Day Engineering Assessment  
| **Date:** September 2026 | **Deliverable:** 1-2 Page Research Note

## 1. The Learner Problem: Why LLD Practice is Broken

Low-Level Design (Object-Oriented Design) is a mandatory hiring gate for Software Development Engineers (SDE-1, SDE-2, and Staff Engineers). In Data Structures and Algorithms (DSA), test vectors provide binary Pass/Fail feedback. In contrast, **LLD evaluation is inherently multi-dimensional, open-ended, and non-deterministic.**

- **Lack of Instant, Explainable Feedback:** In DSA, candidates run code against hidden test cases. In LLD, candidates write classes (e.g. *Parking Lot*, *Elevator*) but have no automated feedback on class coupling, single responsibility violations, interface segregation, and concurrency bottlenecks.
- **Subjectivity vs. Principled Rubrics:** Two engineers can model a *Parking Lot* differently (floor-centric vs. spot-centric) and both can be valid if trade-offs are defended. Learners struggle because existing tutorials show one rigid "solution" without explaining *why* alternative structures fail under evolving requirements.
- **The "Passive Consumption" Trap:** Most candidates practice LLD by watching YouTube videos or reading static GitHub repos without building the active muscle memory of decomposing requirements into clean OOP interfaces.
- **No Version-over-Version Delta Tracking:** Learners rarely know what improved between Attempt #1 and Attempt #2. Without delta scoring (e.g. *" +15 pts on Abstraction; resolved 2 critical coupling concerns "*), candidates cannot track architectural growth.

## 2. Competitive Landscape & Gap Analysis

| Platform / Tool          | Current Workflow                                                                              | Strengths                                                 | Critical Gaps for LLD Learners                                                                                           |
|--------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| LeetCode / HackerRank    | Unit test execution on single function / class interface (e.g. <code>LRUCache.get()</code> ). | Fast deterministic feedback; automated grading.           | <b>Forces algorithmic mindset.</b> Ignores class decomposition, UML relationships, domain modeling, and design patterns. |
| Educative / Grokking LLD | Static reading, reference code walkthroughs, fixed class diagrams.                            | Curated problem statements; good conceptual explanations. | <b>Zero active evaluation.</b> No feedback on learner submissions; passive consumption with no practice loop.            |
| Interviewing.io / Pramp  | Peer or mock human interviews with senior engineers.                                          | Rich, nuanced feedback on design choices and trade-offs.  | <b>High cost (\$150-\$250/session);</b> scheduling friction; not scalable for continuous daily practice.                 |

| Platform / Tool        | Current Workflow                                      | Strengths                                          | Critical Gaps for LLD Learners                                                                                                                                   |
|------------------------|-------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Generic LLMs (ChatGPT) | Pasting code into chat window asking "Review my LLD". | Capable reasoning; instant conversational answers. | <b>Unconstrained, inconsistent output.</b> Generates generic praise/essays without a fixed rubric, lacks AST static validation, and provides no version history. |

### 3. Product Direction: The Hybrid Rubric–Grounded Architecture

To bridge the gap between deterministic test runners and high-cost human mock interviews, **DesignForge** implements a 4-stage active practice loop:

**Structured Practice Loop:**

Problem Catalog → Assumptions & Scope → Mermaid UML Diagram → Polymorphic Code → Design Rationale → Hybrid Evaluator (AST + AI) → Version Delta Comparison

#### Core Product Tenets

- Multi-Faceted Submission:** Learners submit Assumptions, Visual Mermaid UML diagrams, Concrete Code, and Design Rationale.
- Strict 5-Dimension Rubric:** Evaluates Domain Modeling (25%), Abstraction & DIP (25%), Extensibility & GoF Patterns (20%), Edge Cases & Concurrency (15%), and Trade-off Rationale (15%).
- Deterministic + Heuristic + AI Synergy:** AST parser runs static sanity checks in <10ms; Gemini AI or local rule heuristics provide semantic critiques with verbatim code evidence quotes; 100% offline fallback guarantees resilience.
- Iterative Delta Comparison:** Tracks score gains ( $\$Attempt\_N - Attempt\_{\{N-1\}}\$$ ) and highlights resolved architectural smells.